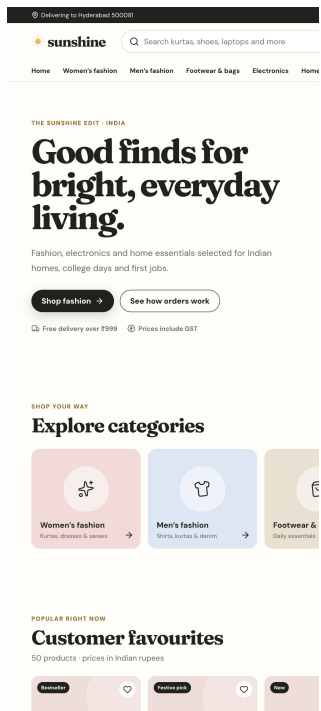


PROJECT REPORT

Sunshine

Indian retail, conversational support, Java order processing and Python services



Author	Divya Rachala
Application	Next.js storefront with Java and Python services
Customer support	Divya conversational shopping assistant
Region	Indian catalogue, pricing and delivery flow
Date	August 2026

A complete shopper journey from product discovery to a traceable order result. All products, payments and order data are synthetic demonstrations.

Project purpose and scope

Sunshine connects the customer journey, service contracts, retail data and quality checks in one application.

Problem statement and root cause

Many student commerce projects use disconnected sample data for the storefront, support, checkout and order history. This makes recommendations and delivery updates difficult to verify. Sunshine uses shared catalogue, cart, stock and order records so the complete customer journey remains consistent.

Project objectives

1. Build a responsive Indian retail experience with prices in INR and familiar delivery expectations.
2. Provide conversational product discovery, size selection, cart support and order tracking.
3. Use Java for typed order processing and explicit failure handling.
4. Use Python for a FastAPI recommendation endpoint and a pandas retail KPI workflow.
5. Deploy a working public application with documented runtime boundaries.

Delivered scope

Area	Delivered capability	Evidence
Catalogue	50 synthetic products in five retail categories	Static data and 50 generated product pages
Customer flow	Search, product, cart, checkout, profile and orders	Browser tested repeat visit paths
Java	Bounded order processing components	Spring Boot and four JUnit scenarios
Python	Recommendation API and retail KPI script	FastAPI, pandas and four pytest checks
Delivery	Docker Compose, CI and Vercel ready UI	Three GitHub Actions jobs

The scope deliberately excludes authentication, a production database and a real payment gateway. These boundaries keep the demonstration safe and reproducible.

From discovery to delivery

The interface follows a realistic retail sequence while keeping every payment and order operation safely simulated.

Step	Customer action	System response
1	Search or choose a category	Filters the 50-product catalogue
2	Ask Divya for a product	Collects style, audience, size and budget
3	Select an option in chat	Adds the verified product and size to the cart
4	Enter address and payment method	Validates Indian mobile and PIN formats
5	Choose a demonstration result	Runs success, decline or stock failure
6	Review profile and recent orders	Persists history and inventory in the browser

Delivering to Hyderabad 500081



[Home](#)
[Women's fashion](#)
[Men's fashion](#)
[Footwear & bags](#)
[Electronics](#)
[Home](#)

SECURE DEMO CHECKOUT

Where should we send your order?

Use sample information only. This portfolio demo does not save your details.

1

Delivery address

Indian PIN codes are supported in this demo.

Full name

Mobile number

Address

City

PIN code

2

Payment method

Retail rules

- Free delivery starts at INR 999. Smaller carts add INR 79.
- UPI, card and cash on delivery are available as simulated methods.
- A successful order reduces browser stock. A decline keeps stock available for retry.

One customer interface, clear service boundaries

The Next.js application connects conversational support, Java order processing, Python recommendations and hosted adapters through validated contracts.

Layer	Technology	Responsibility
Browser	React and TypeScript	Interactive catalogue, cart and checkout state
Web app	Next.js 16	Pages, metadata, route handlers and Vercel deployment
Customer support	Ollama Cloud and TypeScript	Intent selection and verified commerce actions
Order service	Java 17 and Spring Boot 4	Validation and five component orchestration
Recommendation service	Python 3.13 and FastAPI	Content based product ranking
Analytics	pandas	Raw order CSV to KPI JSON
Delivery	Docker and GitHub Actions	Reproducible local stack and automated checks

Runtime sequence

1. The customer uses the Next.js storefront in a browser.
2. The storefront sends conversational requests to the customer support route.
3. Ollama can interpret the request, while Sunshine provides verified product, cart and order data.
4. Order requests use Spring Boot or the contract compatible hosted adapter.
5. Recommendation requests use FastAPI or the contract compatible hosted adapter.
6. The pandas workflow transforms raw order records into reproducible KPI output.

Deployment adapter pattern

If `JAVA_BACKEND_URL` or `PYTHON_BACKEND_URL` is present, Next.js proxies to the real service. If a service is unavailable, a TypeScript adapter applies the same deterministic contract so the Vercel demo remains usable. The response identifies the source, which allowed integration tests to prove that Java and Python were actually called locally.

A student scale multi agent order workflow

Spring Boot is the canonical order service. OrderOrchestrator calls five deterministic agents in sequence and stops safely when a prerequisite fails.

Agent	Input	Decision	Output
Catalogue	Cart + stock	Can stock be reserved?	Reservation or OUT_OF_STOCK
Risk	Order context	Do rules pass?	Approved decision
Payment	Reserved order	Can payment proceed?	Reference or failure
Fulfilment	Eligible order	When can it arrive?	Shipment plan
Notification	Final result	Which update applies?	History event

How the orchestration works

Each agent is an ordinary Java component with one responsibility. The orchestrator passes typed results through the sequence and records completed, failed or skipped steps. These order agents do not use a language model. Divya is the separate customer facing AI assistant.

Scenario behaviour

Scenario	Catalogue	Risk	Payment	Delivery	Notify
SUCCESS	done	done	done	done	done
PAYMENT_FAILED	done	done	failed	skipped	done
OUT_OF_STOCK	failed	skipped	skipped	skipped	done

Service verification

Local integration testing confirmed that the Java API handled the order request and returned the x-sunshine-service value java. The hosted adapter returns the value vercel-adapter when the external Java service is not configured.

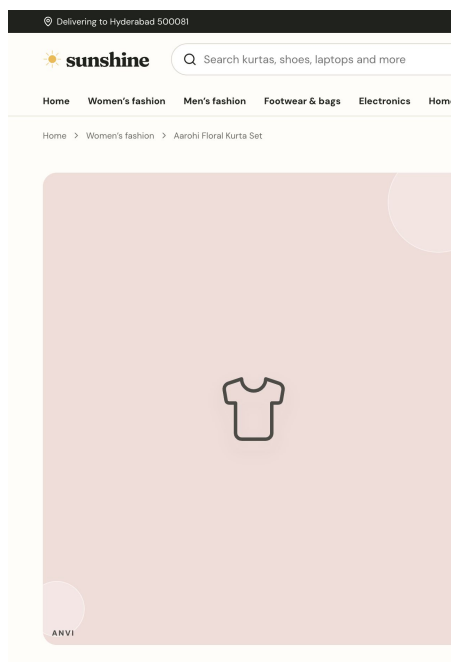
Recommendations and measurable retail KPIs

Python has two focused roles: ranking related products through FastAPI and transforming a raw order CSV through pandas.

Recommendation method

The service receives a reference product and compatible candidates from Next.js. It filters to the same category and ranks by rating and relative price distance. This is intentionally content based because the demo does not collect customer histories.

score = rating multiplied by 2, reduced by absolute price difference divided by reference price

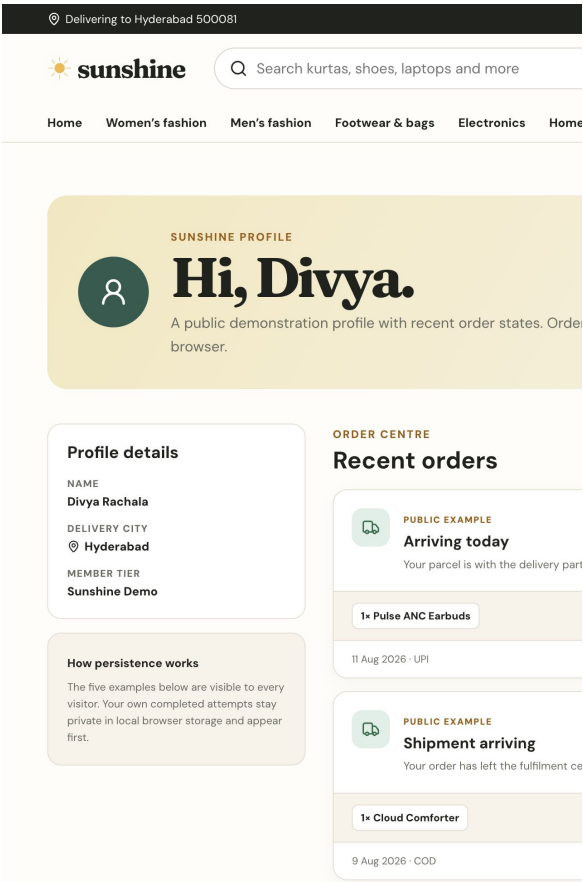


KPI pipeline output

KPI	Result
Orders received	12
Orders confirmed	9
Confirmation rate	75.0%
Confirmed revenue	INR 37,136
Average order value	INR 4,126.22
Top revenue city / category	Delhi / Electronics

Conversational support and repeat visit state

Sunshine combines an original retail identity with a profile menu, Divya support, seeded order examples and private browser activity.



Decision	Reason
Warm cream, yellow and green palette	Creates a recognisable Sunshine identity without imitating Amazon
Seeded order lifecycle examples	Every visitor can inspect arriving, shipped, delivered and failed states
Versioned browser storage	Personal orders and stock changes persist privately across visits
Low stock product rules	The final unit becomes unavailable when the shopper returns
Conversational product tools	Verified catalogue options can be added to cart with a selected size
Responsive grids	Profile and orders collapse cleanly on small screens

Accessibility checks include labelled search, semantic navigation, form labels, radio groups, keyboard accessible buttons, visible focus styles and readable success and failure text that does not depend only on colour.

A failed payment is a designed outcome

The application explains what happens when an operation does not succeed instead of hiding the result behind a generic error message.

Customer progress	Successful order	Payment failed	Item unavailable
Items checked	Available	Available	Unavailable
Payment reviewed	Accepted	Declined	Not started
Delivery update	Estimate created	Not created	Not created
Cart result	Cleared	Available for retry	Unavailable item remains visible
Recent orders	Confirmed	Payment failed	Item unavailable

Observed behaviour

1. The catalogue reservation completes before payment begins.
2. Payment returns a method specific explanation when authorisation fails.
3. Delivery planning does not run when payment eligibility is not met.
4. No payment reference or delivery date is generated.
5. The cart remains available so the customer can retry without rebuilding it.
6. The failure is recorded in the visitor's recent order history.

This trace turns a failure into evidence of control flow, state handling and customer communication.

Automated checks across the complete stack

Tests target business behaviour rather than only verifying that files compile.

Quality gate	Coverage	Result
Vitest	Pricing, chat, order lookup and stock aware paths	12 passed
JUnit	Success, decline, scenario and real stock exhaustion	4 passed
pytest	Ranking category, determinism, API and validation	4 passed
ESLint	React and TypeScript quality rules	Passed
Next.js build	Routes, chat, profile and 50 product paths	63 pages generated
Browser QA	Profile, last unit, return visit and failures	Passed
Integration	Next.js to Java and Next.js to Python	Both sources confirmed

Continuous integration

GitHub Actions runs three independent jobs. The frontend job installs Node 22 dependencies, tests, lints and builds. The Java job sets up Temurin 17 and runs Maven Wrapper tests. The Python job installs versioned requirements and runs pytest.

Deployment

1. Docker Compose starts the Next.js, Java and Python services as one local system.
2. Vercel hosts the public Next.js interface and compatible route adapters.
3. Environment variables switch the route handlers to external Java and Python services without browser changes.
4. A private Ollama key enables language understanding without exposing the credential to the browser.

Environment variable	Purpose
JAVA_BACKEND_URL	Proxy order requests to Spring Boot
PYTHON_BACKEND_URL	Proxy recommendation scoring to FastAPI
OLLAMA_API_KEY	Enable Ollama Cloud intent selection

Security boundary

The demo never requests real card, UPI, password or customer identity data. The checkout form explicitly asks for sample information and does not persist delivery details.

Implemented capabilities and future scope

Sunshine connects customer state, conversational support, service contracts, failure rules and reproducible outputs in one application.

Implemented capabilities

Area	Implementation
React state	Versioned cart, order and inventory stores with useSyncExternalStore
Next.js	Server rendered product paths plus small client boundaries
Java design	Five constructor injected components coordinated by OrderOrchestrator
Conversational AI	Ollama intent selection with verified product, cart and order actions
Python API	Typed FastAPI request model and deterministic ranking
Data analysis	CSV to JSON KPI pipeline with pandas
Failure handling	Early exit, skipped steps and retry safe cart state
Deployment	Contract preserving adapter for platform runtime limits

Limitations

1. Synthetic data does not represent real customer demand or production scale performance.
2. The application has no authentication, shared order database or live stock source.
3. Content based ranking does not learn from user behaviour.
4. Illustrative processing durations are trace labels, not benchmarks.

Reasonable next steps

1. Add PostgreSQL order persistence and idempotency keys.
2. Deploy Spring Boot and FastAPI to suitable container runtimes and configure Vercel URLs.
3. Introduce a real event log and an observable correlation ID across services.
4. Replace synthetic recommendations only when consented behavioural data is available.

Sunshine is a complete student scale project with clear technical and data boundaries.

Divya Rachala | github.com/divyarachala1812